

A Monte Carlo Path-Tracer for CSE306

Project report – Labs 1 to 4

Vlad Andreescu | CSE306 Computer Graphics | May 18, 2026

1 Introduction

I acknowledge the use of AI in this report as a tool solely serving the purpose of polishing certain phrasings and occasionally speeding up the writing of LaTeX math formulas taken from the course notes. AI was also used as a means to ask technical questions during the labs (with authorization from the TA).

This report documents the design and implementation of a Monte Carlo path-tracer. The implementation covers the full set of features required by the project specification: a pinhole camera with ray–sphere intersection, Lambertian and perfectly specular (mirror) materials, direct illumination from a point light source with hard shadows, indirect illumination through recursive Monte Carlo path sampling without Russian roulette, stochastic antialiasing, ray–mesh intersection for arbitrary triangulated geometry, and a Bounding Volume Hierarchy (BVH) to accelerate the latter. In addition, two optional features are implemented: dielectric transparency with total internal reflection (Lab 2), and diffuse texture mapping (Lab 4).

2 Lab 1 – Direct Illumination on Spheres

2.1 Camera model and ray generation

The scene is observed through a pinhole camera placed at $O_{\text{cam}} = (0, 0, 55)$ and looking towards $-z$ with a horizontal field of view $\alpha = 60^\circ$. For an image of width W and height $H = W$, the focal distance expressed in pixel units is $d = (W/2)/\tan(\alpha/2)$. For each pixel (i, j) a primary ray $r(t) = O_{\text{cam}} + tu$ is cast with unnormalised direction

$$u \propto (j - W/2 + 0.5, H/2 - i - 0.5, -d),$$

which is then normalised to unit length. The $+0.5$ offsets place the ray through the centre of each pixel and the second component is flipped so that increasing image rows correspond to decreasing world y .

2.2 Ray–sphere intersection

A sphere of centre C and radius R is intersected by writing $\|O + tu - C\|^2 = R^2$ and solving the resulting quadratic in t . With $\Delta = \langle u, O - C \rangle^2 - (\|O - C\|^2 - R^2)$ the two candidate parameters are $t_{1,2} = -\langle u, O - C \rangle \mp \sqrt{\Delta}$. The implementation discards the case $\Delta < 0$, returns the smaller positive root when one exists, and otherwise reports no intersection. The hit point and outward unit normal are then $P = O + tu$ and $N = (P - C)/R$. A scene-wide intersection routine loops over all objects and keeps the hit with the smallest strictly positive t .

2.3 Direct illumination with shadows

For a Lambertian (diffuse) surface of albedo $\rho \in [0, 1]^3$ illuminated by an isotropic point light of intensity I located at S , the outgoing radiance in the camera direction is

$$L_o(P) = \frac{I}{4\pi\|S - P\|^2} \frac{\rho}{\pi} \max(0, \langle N, \omega_i \rangle),$$

where $\omega_i = (S - P)/\|S - P\|$ is the unit vector from the surface point to the light. The factor $1/(4\pi d^2)$ accounts for the inverse-square falloff from an omnidirectional source, while ρ/π is the BRDF of an energy-conserving Lambertian material.

Hard shadows are obtained by tracing a secondary ray from P towards S before evaluating the direct lighting term. If this shadow ray hits any object at a parameter smaller than the distance to the light,

P is occluded and contributes zero direct radiance. To avoid self-intersection caused by floating-point round-off on P , the shadow ray origin is offset along the surface normal by $\varepsilon = 10^{-4}$.

2.4 Mirror surfaces

Perfectly specular surfaces are handled by a separate code path in the shading routine. When a ray with direction ω_i hits a mirror at P with normal N , the reflected direction $\omega_r = \omega_i - 2\langle\omega_i, N\rangle N$ is computed and `getColor` is invoked recursively on the new ray $P + \varepsilon N + t\omega_r$. Recursion is bounded by a maximum depth parameter (set to 5 in all our experiments), beyond which the accumulated radiance is taken to be zero. The same ε offset as for shadow rays is applied to suppress acne artefacts.

2.5 Gamma correction

Radiance values computed by the path tracer live in linear space. Before being quantised to 8-bit sRGB output, they are mapped through the standard gamma curve $c \mapsto c^{1/2.2}$. This compression is applied only at the moment of writing the PNG file: all intermediate radiance arithmetic remains linear, which is essential for the indirect-lighting computations introduced in Lab 2.

2.6 Results

Figure 1 shows the cumulative output of the renderer at the end of Lab 1. The scene contains two test spheres of identical radius and albedo placed on either side of the optical axis, plus six large coloured spheres that play the role of walls, ceiling and floor. The left sphere is purely diffuse and exhibits the expected Lambertian falloff: the side facing the off-axis point light is brightest, the angle-of-incidence cosine progressively darkens the upper-right portion, and a sharp cast shadow appears on the floor. The right sphere is flagged as a perfect mirror: its surface reproduces the surrounding coloured walls, the ceiling, the floor, and, in the lower-left of its hemisphere, the reflected silhouette of the diffuse sphere. The fact that both spheres cast distinct, geometrically consistent shadows confirms that the shadow ray and $+\varepsilon N$ offset logic operate correctly for both material types. Refraction is not implemented in Lab 1 (the prompt lists it as an optional “if possible” item), but the transparency code path is added in Lab 2 and illustrated there.

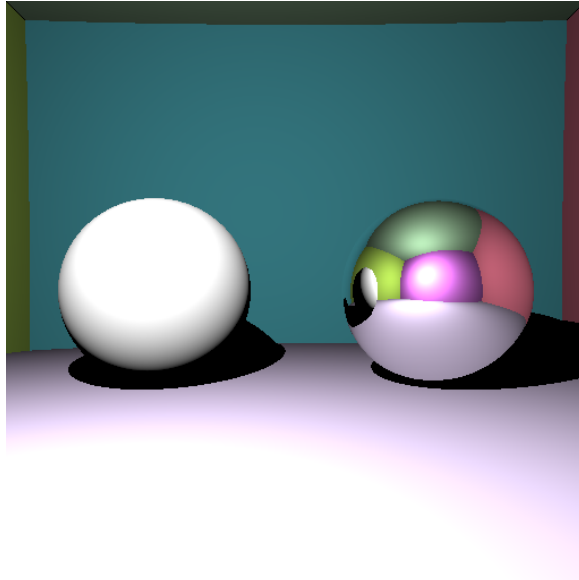


Figure 1: Lab 1 output. Diffuse sphere (left) and mirror sphere (right) of common radius 10, illuminated by a single off-axis point light. Hard shadows and Lambertian falloff are visible on both spheres; the mirror reflects the surrounding walls and the diffuse sphere on the opposite side of the scene. Resolution 512×512 , one sample per pixel, maximum recursion depth 5. Render time: ≈ 90 ms on a modern multi-core laptop (OpenMP enabled).

3 Lab 2 – Indirect Illumination and Antialiasing

3.1 Recursive path-traced indirect illumination

Lab 1 only accounts for light arriving directly at the camera from the point source. Lab 2 augments the shading routine with an indirect-illumination term that gathers radiance arriving at P from all other surfaces of the scene. Following the rendering equation restricted to Lambertian surfaces, the outgoing radiance from P becomes

$$L_o(P) = L_{\text{dir}}(P) + \frac{\rho}{\pi} \int_{\Omega^+} L_i(P, \omega) \langle N, \omega \rangle d\omega,$$

where Ω^+ is the upper hemisphere about N . The hemispherical integral is estimated by a single Monte Carlo sample drawn according to a cosine-weighted distribution, whose probability density is $p(\omega) = \langle N, \omega \rangle / \pi$. With this importance sampling the unbiased one-sample estimator simplifies dramatically:

$$\frac{1}{p(\omega)} \frac{\rho}{\pi} L_i(P, \omega) \langle N, \omega \rangle = \rho \cdot L_i(P, \omega),$$

which is exactly the form used in the implementation (component-wise product of the albedo with the recursively gathered radiance). The cosine-weighted direction is generated in a local frame around N using the standard Malley method: (r_1, r_2) uniform in $[0, 1]^2$ yields

$$(\cos(2\pi r_1)\sqrt{1-r_2}, \sin(2\pi r_1)\sqrt{1-r_2}, \sqrt{r_2})$$

in the local hemisphere. The frame (T_1, T_2, N) is built by zeroing out the smallest-magnitude component of N , which avoids the near-singular case where a naive cross product with a fixed axis would be ill-conditioned.

Recursion is the same as for mirrors and is bounded by `max_light_bounce` = 5. Russian roulette is intentionally not used, as it is explicitly excluded from the project specification.

3.2 Antialiasing by stochastic supersampling

Aliasing produced by single-sample-per-pixel rendering is removed by drawing $N_s = 32$ primary rays per pixel and averaging their contributions. Rather than offsetting each sample uniformly within the pixel square, we generate an isotropic Gaussian jitter of standard deviation $\sigma = 0.5$ pixel using the Box–Muller transform on the two uniform variates r_1, r_2 :

$$(dx, dy) = \sigma \sqrt{-2 \ln r_1} (\cos(2\pi r_2), \sin(2\pi r_2)).$$

This yields a soft pixel reconstruction filter rather than a hard box, and removes the visible staircase pattern on the silhouette of the spheres at essentially no additional cost. Each thread owns its own `std::default_random_engine`, indexed through `omp_get_thread_num()`, so that parallel sampling is deterministic and free of contention.

3.3 Transparent (refractive) surfaces (optional feature)

Beyond the features required by the Lab 2 brief (cosine-weighted indirect illumination and antialiasing), dielectric transparency was also implemented as a bonus. When a ray hits an object flagged as `transparent`, the indices of refraction on either side are assigned according to whether the ray is entering or leaving the medium (detected from the sign of $\langle \omega_i, N \rangle$); the inward-facing normal is flipped accordingly. Snell's law gives

$$\sin^2 \theta_t = \left(\frac{n_1}{n_2} \right)^2 (1 - \cos^2 \theta_i),$$

and when $\sin^2 \theta_t > 1$ the ray undergoes total internal reflection and is treated as a mirror. Otherwise the refracted direction $\omega_t = \frac{n_1}{n_2} \omega_i - \left(\frac{n_1}{n_2} \cos \theta_i + \cos \theta_t \right) N$ is computed and the recursive call is made from P offset by $-\varepsilon N$ (i.e. into the new medium) to avoid self-intersection. Fresnel coefficients are not used: the entire energy is sent along the transmitted (or totally reflected) ray.

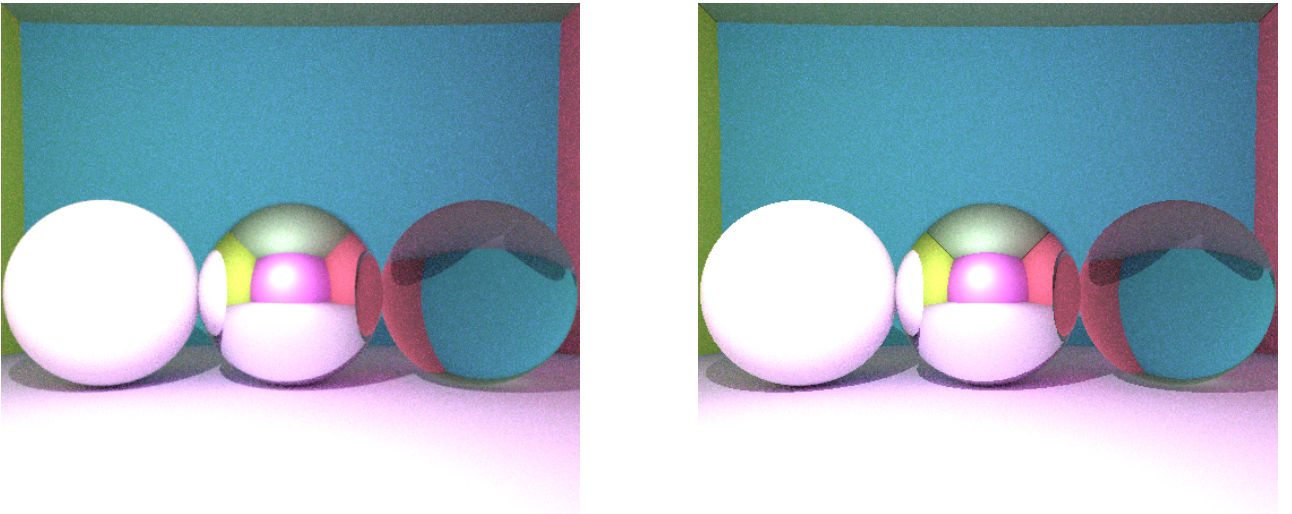
3.4 Results

Figure 2 shows the cumulative output at the end of Lab 2 alongside an otherwise identical render produced with the sub-pixel jitter disabled. The comparison isolates the effect of antialiasing: any difference between the two panels can only be attributed to the stochastic supersampling of the pixel filter, since the scene, the indirect-illumination integrator, the random seeds and the number of samples per pixel are kept fixed.

The contribution of indirect illumination is most readily seen on the lower half of the back wall and on the floor in front of the spheres, both of which receive a soft mauve cast bled from the purple back wall, and on the underside of the diffuse sphere, which in Lab 1 would have been fully black where it is shadowed from the point light. The ceiling, which is not directly lit at all, shows a gentle gradient produced entirely by light bouncing once off the floor and walls. These regions are visible in both panels and are the direct evidence that the recursive Monte Carlo gather is doing its job.

The contribution of antialiasing is more subtle (zoom in is required) but clearly visible on the silhouettes of the spheres against the back wall, and along the diagonal corners where two coloured walls meet. In the no-AA panel (right) the silhouettes of the spheres exhibit a staircase aliasing pattern, and the wall-wall corners show pixel-level discretisation: in the AA panel (left) the same edges appear smooth, because the Gaussian-distributed jitter implicitly convolves the image with a soft reconstruction filter. The rest of the two panels is visually identical, confirming that AA only affects edge reconstruction and does not alter the underlying shading.

The scene also illustrates the three material types implemented in the renderer: diffuse (left), perfect mirror (centre, reflecting the surrounding walls) and the transparent dielectric (right, refracting the back wall through its centre and showing a darker rim where rays exit close to the grazing angle). Some residual Monte Carlo noise remains visible at $N_s = 32$, mostly on the ceiling and in the shadowed regions, and would decrease as $1/\sqrt{N_s}$ with more samples.



(a) With antialiasing (Gaussian jitter, $\sigma = 0.5$ px).

(b) Without antialiasing (jitter disabled).

Figure 2: Lab 2 output. Both renders use 32 samples per pixel, maximum depth 5, and resolution 512×512 ; the only difference is the sub-pixel jitter, which is enabled on the left and forced to zero on the right. The AA effect is most easily observed on the silhouettes of the spheres against the back wall and on the diagonal where two coloured walls meet. Render time per image: ≈ 2 s on a modern multi-core laptop (OpenMP enabled).

4 Lab 3 – Ray–Mesh Intersection

4.1 The Möller–Trumbore intersection algorithm

To render arbitrary triangulated models, the raytracer is extended to compute intersections between rays and triangles. Given a triangle defined by its three vertices A, B, C , any point on its surface can

be written in barycentric coordinates as $P = \alpha A + \beta B + \gamma C$ with $\alpha + \beta + \gamma = 1$. Substituting the ray equation $P(t) = O + tu$ and eliminating α yields the linear system

$$-tu + \beta(B - A) + \gamma(C - A) = O - A.$$

Introducing the edge vectors $e_1 = B - A$, $e_2 = C - A$ and the geometric triangle normal $\vec{n} = e_1 \times e_2$, the system is solved by the Möller–Trumbore scalar-triple-product formulae,

$$\beta = \frac{\langle e_2, (A - O) \times u \rangle}{\langle u, \vec{n} \rangle}, \quad \gamma = -\frac{\langle e_1, (A - O) \times u \rangle}{\langle u, \vec{n} \rangle}, \quad t = \frac{\langle A - O, \vec{n} \rangle}{\langle u, \vec{n} \rangle},$$

with $\alpha = 1 - \beta - \gamma$. Rays nearly parallel to the triangle plane (i.e. $|\langle u, \vec{n} \rangle| < 10^{-12}$) are rejected. A valid intersection requires $\alpha, \beta, \gamma \geq 0$ and a strictly positive t smaller than the closest hit found so far.

4.2 Bounding box early rejection

Testing every primary, shadow and indirect ray against every triangle of a complex mesh is prohibitive. Before introducing a full Bounding Volume Hierarchy in Lab 4, Lab 3 already accelerates the process by wrapping the entire mesh in a single global Axis-Aligned Bounding Box (AABB), built once from the extremal vertex coordinates.

The box is defined by two extrema $B_{\min}$ and $B_{\max}$, and is intersected by the slab method. For each spatial axis $k \in \{x, y, z\}$, the ray meets the two parallel slab planes at

$$t_0^k = \frac{B_{\min}^k - O^k}{u^k}, \quad t_1^k = \frac{B_{\max}^k - O^k}{u^k}.$$

After ordering so that $t_0^k \leq t_1^k$, the latest entry $t_{\text{entry}} = \max_k t_0^k$ and the earliest exit $t_{\text{exit}} = \min_k t_1^k$ are accumulated. The ray hits the box if and only if $t_{\text{entry}} \leq t_{\text{exit}}$ and $t_{\text{exit}} > 0$; otherwise the whole mesh is rejected in a handful of arithmetic operations.

4.3 Barycentric normal interpolation

To prevent the mesh from appearing as a collection of sharply faceted polygons, Phong normal interpolation is used. When a valid hit is found, the barycentric coordinates (α, β, γ) already computed by Möller–Trumbore are reused to interpolate the per-vertex normals provided by the `.obj` file:

$$N(P) = \alpha N_A + \beta N_B + \gamma N_C,$$

which is then renormalised before being returned to the shading routine, creating the illusion of a smooth surface over a coarse triangulation. A flat-shading fallback (the geometric normal $\vec{n}$) is used for triangles whose `.obj` record does not carry vertex normals.

4.4 Results

Figure 3 shows the `cat.obj` model integrated into the Cornell-box scene built in earlier labs. The successful rendering of the complex geometry confirms the correctness of the Möller–Trumbore implementation. More importantly, the image demonstrates how the new mesh primitive interacts with the rest of the renderer. First, despite the underlying geometry being a mesh of flat triangles, the surface of the cat appears continuously curved, which directly validates the barycentric normal interpolation described above. Second, the cat casts a geometrically consistent hard shadow on the back wall and the floor, and its underside is softly illuminated by light bouncing off the floor and walls, which proves that shadow rays (with the $+\varepsilon N$ offset) and the recursive cosine-weighted indirect rays from Lab 2 intersect the mesh just as reliably as they intersect the analytical spheres.

This scene contains 3954 triangles. With only a global AABB and no internal acceleration structure, rendering it at 32 spp took approximately 180 s. Almost all of that time is spent in the linear scan over the triangle array for every ray that enters the mesh’s bounding box; replacing this scan with a BVH is the subject of Lab 4.

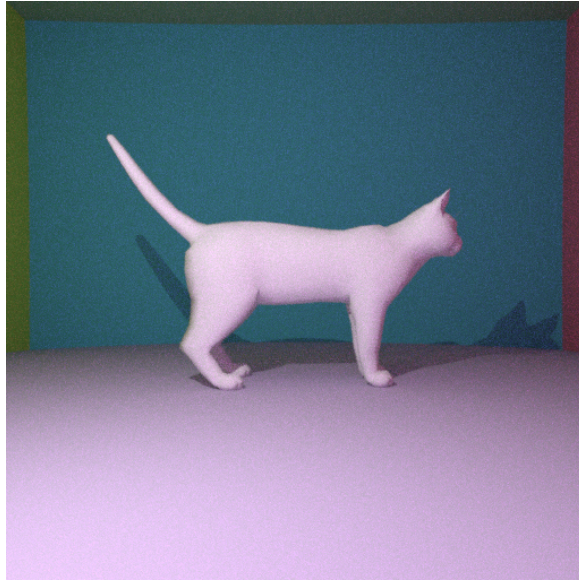


Figure 3: Lab 3 output. A triangulated cat mesh loaded from an `.obj` file, rendered using the Möller–Trumbore algorithm and bounded by a single global AABB. Smooth shading is obtained via barycentric interpolation of vertex normals. The mesh fully participates in direct shadows and recursive indirect illumination. Resolution 512×512 , 32 samples per pixel, maximum depth 5. Triangles: 3954. Render time: ≈ 180 s.

5 Lab 4 – Bounding Volume Hierarchy

5.1 BVH construction and spatial partitioning

While the global bounding box from Lab 3 prevents intersection tests when a ray misses the mesh entirely, every ray that does enter the box must still be evaluated against all 3954 triangles. Lab 4 introduces a binary Bounding Volume Hierarchy (BVH) built over the triangle array. Each `BVHNode` stores its own axis-aligned bounding box ($\vec{b}_{\min}, \vec{b}_{\max}$), pointers to its `left` and `right` children, and a half-open primitive range `[start_index, end_index)` into the mesh’s triangle list.

The hierarchy is constructed top-down by a recursive routine. At each step, the bounding box of the current sub-range is computed from the vertices of its triangles. If the range contains 5 triangles or fewer, the recursion terminates and the node becomes a leaf. Otherwise, the partitioning axis is taken to be the longest one of the current bounding box,

$$\vec{d} = \vec{b}_{\max} - \vec{b}_{\min}, \quad \text{axis} = \arg \max_k d_k,$$

and the split plane is placed at the midpoint along that axis,

$$x_{\text{mid}} = \frac{1}{2}(b_{\min}^{\text{axis}} + b_{\max}^{\text{axis}}).$$

The triangles in the range are then reordered in-place with a quicksort-style partition: the sorting key for each triangle is the coordinate of its centroid along the split axis, $x_c = \frac{1}{3}(A^{\text{axis}} + B^{\text{axis}} + C^{\text{axis}})$, and triangles whose centroid lies below x_{mid} are moved to the beginning of the range. To avoid infinite recursion on degenerate distributions where all centroids fall on the same side of the split, the node is converted into a leaf whenever the pivot ends up at either boundary. Otherwise, two child nodes are allocated and the construction recurses on the two sub-ranges.

5.2 Iterative depth-first traversal

Ray-mesh intersection through the BVH replaces deep recursive function calls by an explicit traversal stack of fixed size 64. The traversal behaves as a standard depth-first search:

1. The root node is pushed onto the stack.
2. While the stack is not empty, the top node is popped.

3. If the popped node is a leaf, the ray is tested against every triangle in $[\text{start_index}, \text{end_index})$ using Möller–Trumbore, conditionally updating the closest hit distance t_{closest} and the index of the active triangle.
4. Otherwise, for each of the node’s two children, the slab test of Lab 3 is applied to the child’s bounding box, and only children whose box is hit by the ray are pushed onto the stack.

Sub-trees whose root bounding box is missed by the ray are therefore never visited at all, which is what produces the asymptotic speedup over the linear Lab 3 scan.

5.3 Diffuse texture mapping (optional feature)

As an additional bonus feature on top of the required BVH, diffuse texture mapping was implemented for triangulated meshes. When a hit is registered on a textured triangle, the per-vertex (u, v) coordinates loaded from the `.obj` file are interpolated using the same barycentric coordinates returned by Möller–Trumbore,

$$u = \alpha u_A + \beta u_B + \gamma u_C, \quad v = \alpha v_A + \beta v_B + \gamma v_C,$$

and passed to a `sampleTexture` routine. The latter implements wrap-around tiling by fractional extraction $u \leftarrow u - \lfloor u \rfloor$ (and likewise for v), and a v -flip that converts the OBJ convention ($v = 0$ at the bottom) into the row-major image convention used by `stb_image`. Because the texture file (`cat_diff.png`) is stored in 8-bit sRGB, its pixel values are first divided by 255 and then gamma-decoded into linear radiance space,

$$c_{\text{linear}} = ((R/255)^{2.2}, (G/255)^{2.2}, (B/255)^{2.2}),$$

before being injected as the local albedo in the path-tracing pipeline. The final image-space gamma correction is therefore applied only once, on the accumulated linear radiance buffer at the moment of writing the PNG, exactly as in Lab 1.

5.4 Results

Figure 4 shows the final rendered output combining all mandatory and optional features. The mesh that previously displayed a uniform grey albedo now resolves the structural variations stored in the texture map: the orange tabby fur, the darker stripes along the back and tail, the lighter chest and paws, and the pink nose are all reproduced from the sampled diffuse texture. The sharpness of these patterns across the animal’s silhouette confirms that the barycentric UV interpolation is consistent with the Möller–Trumbore intersection loop.

The remaining visual content is consistent with everything verified in earlier labs: the textured body exhibits Lambertian falloff, casts a single hard shadow on the back wall and the floor, and receives indirect bounces from the coloured walls of the Cornell box.

The performance impact of the BVH is dramatic. By replacing the linear $O(N)$ scan over triangles with an $O(\log N)$ tree traversal, the total rendering time of this identical scene collapses from 180 s in Lab 3 to just 4 s in Lab 4, a $\approx 45\times$ speedup on a 3954-triangle mesh.

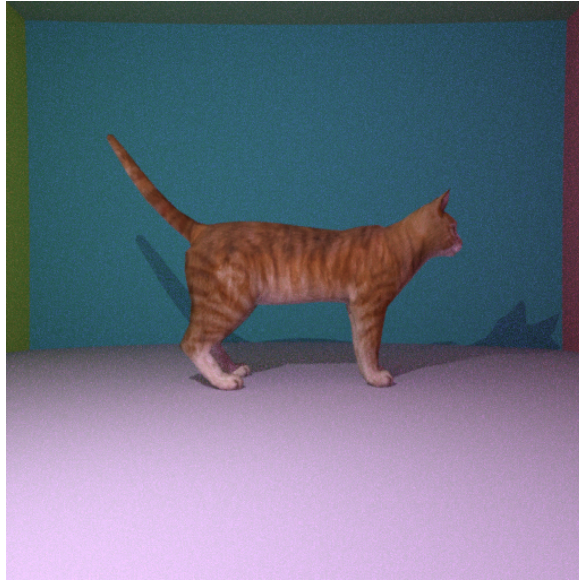


Figure 4: Lab 4 output. The same cat mesh as in Figure 3, but now traversed through a binary BVH with longest-axis midpoint splitting and a leaf threshold of 5 triangles. The diffuse albedo is now read from a (u, v) -mapped sRGB texture decoded to linear space at sampling time. Resolution 512×512 , 32 samples per pixel, maximum depth 5. Triangles: 3954. Render time: ≈ 4 s.

6 Conclusion

This project develops a complete Monte Carlo path-tracer that fulfills every requirement of the project specification: diffuse and mirror materials, direct illumination with hard shadows from a point light source, indirect illumination via cosine-weighted hemispheric sampling without Russian roulette, stochastic antialiasing, ray-mesh intersection through Möller-Trumbore with Phong-interpolated normals, and a binary Bounding Volume Hierarchy that reduces the dominant per-ray cost from linear to logarithmic in the triangle count. On the 3954-triangle cat scene, the BVH alone accounts for a $\approx 45\times$ speedup (from 180 s to 4 s) at constant visual quality, which is the single most consequential optimisation introduced in the project.

Two optional features are also implemented and illustrated: dielectric transparency with total internal reflection (Lab 2), and gamma-aware diffuse texture mapping over the triangle mesh (Lab 4). Together with the required features, these were sufficient to render the fully textured, globally illuminated cat scene of Figure 4 in a few seconds on a single multi-core laptop.